

CCES – College of Computer Engineering and Sciences

CS3401 – Algorithm Design and Analysis

Chapter 3- Brute Force & Exhaustive Search

References:

- ❖ *Introduction to the Design and Analysis of Algorithms. Anany V. Levitin, Pearson 3rd Edition.*

Brute Force

Introduction



جامعة الأمير سطام بن عبدالعزيز
PRINCE SATTAM BIN ABDULAZIZ UNIVERSITY

Straightforward way to solve a problem, based on the definition of the problem itself. Often involves checking all possibilities.

- Pros:
 - widely applicable
 - easy
 - good for small problem sizes
- Con:
 - often inefficient for large inputs

Brute Force

Introduction

Examples:

- Computing a^n ($a > 0$, n a nonnegative integer)

Multiplying 1 by a n times $a^n = \underbrace{a \times a \times \cdots \times a}_{n \text{ times}} \Rightarrow O(n)$

- Find the sum of all natural numbers till n

Enumerate all numbers from 1 to n But this is clearly not the cheapest solution since

$$\sum_{i=1}^n i = \frac{n \times (n+1)}{2}$$

Brute Force

Sorting

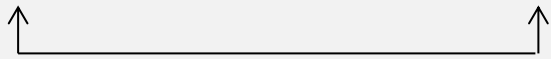
- Selection sort

Scan the array to find its smallest element and swap it with the first element.

Then, starting with the second element, scan the elements to the right of it to find the smallest among them and swap it with the second elements.

Generally, on pass i ($0 \leq i \leq n-2$), find the smallest element in $A[i..n-1]$ and swap it with $A[i]$:

$A[0] \leq \dots \leq A[i-1] \mid A[i], \dots, A[\min], \dots, A[n-1]$
in their final positions



Brute Force

Sorting



جامعة الأمير سطام بن عبدالعزيز
PRINCE SATTAM BIN ABDULAZIZ UNIVERSITY

- Selection sort

ALGORITHM *SelectionSort*($A[0..n - 1]$)

//Sorts a given array by selection sort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n - 2$ **do**

$min \leftarrow i$

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[j] < A[min]$ $min \leftarrow j$

 swap $A[i]$ and $A[min]$

89	45	68	90	29	34	17
17	45	68	90	29	34	89
17	29	68	90	45	34	89
17	29	34	90	45	68	89
17	29	34	45	90	68	89
17	29	34	45	68	90	89
17	29	34	45	68	89	90

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n - 1 - i) = \frac{(n - 1)n}{2}. \quad \Rightarrow \quad \Theta(n^2)$$

Brute Force

Sorting



جامعة الأمير سطام بن عبدالعزيز
PRINCE SATTAM BIN ABDULAZIZ UNIVERSITY

- Bubble sort
 - scan array, swapping out-of-order neighbors
 - continue until no swaps are needed

ALGORITHM *BubbleSort*($A[0..n - 1]$)

//Sorts a given array by bubble sort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow 0$ **to** $n - 2 - i$ **do**

if $A[j + 1] < A[j]$ swap $A[j]$ and $A[j + 1]$

89	↔ [?]	45		68		90		29		34		17
45		89	↔ [?]	68		90		29		34		17
45		68		89	↔ [?]	90	↔ [?]	29		34		17
45		68		89		29		90	↔ [?]	34		17
45		68		89		29		34		90	↔ [?]	17
45		68		89		29		34		17		90
45	↔ [?]	68	↔ [?]	89	↔ [?]	29		34		17		90
45		68		29		89	↔ [?]	34		17		90
45		68		29		34		89	↔ [?]	17		90
45		68		29		34		17		89		90

etc.

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 = \sum_{i=0}^{n-2} [(n-2-i) - 0 + 1]$$

$$= \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2} \in \Theta(n^2).$$

Brute Force

String Matching

- pattern: a string of m characters to search for
- text: a (longer) string of n characters to search in
- problem: find a substring in the text that matches the pattern

ALGORITHM *BruteForceStringMatch*($T[0..n - 1]$, $P[0..m - 1]$)

//Implements brute-force string matching

//Input: An array $T[0..n - 1]$ of n characters representing a text and

// an array $P[0..m - 1]$ of m characters representing a pattern

//Output: The index of the first character in the text that starts a

// matching substring or -1 if the search is unsuccessful

for $i \leftarrow 0$ **to** $n - m$ **do**

$j \leftarrow 0$

while $j < m$ **and** $P[j] = T[i + j]$ **do**

$j \leftarrow j + 1$

if $j = m$ **return** i

return -1

Efficiency: $O(NM)$, $\Theta(N)$

N O B O D Y _ N O T I C E D _ H I M
N O N T O N T O N T O N T O N T O N
N O N T O N T O N T O N T O N T O N

Brute Force

Closest Pair

- Find the two closest points in a set of n points (in the two-dimensional Cartesian plane).
- we consider only the pairs of points (p_i, p_j) for which $i < j$ to avoid computing the distance between the same pair twice

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

$$\begin{aligned} C(n) &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n 2 = 2 \sum_{i=1}^{n-1} (n - i) \\ &= 2[(n - 1) + (n - 2) + \dots + 1] = (n - 1)n \in \Theta(n^2). \end{aligned}$$

ALGORITHM *BruteForceClosestPair(P)*

//Finds distance between two closest points in the plane by brute force

//Input: A list P of n ($n \geq 2$) points $p_1(x_1, y_1), \dots, p_n(x_n, y_n)$

//Output: The distance between the closest pair of points

$d \leftarrow \infty$

for $i \leftarrow 1$ **to** $n - 1$ **do**

for $j \leftarrow i + 1$ **to** n **do**

$d \leftarrow \min(d, \text{sqrt}((x_i - x_j)^2 + (y_i - y_j)^2))$ //sqrt is square root

return d



improvement

ignore the square-root function and
compare the values

$$(x_i - x_j)^2 + (y_i - y_j)^2$$

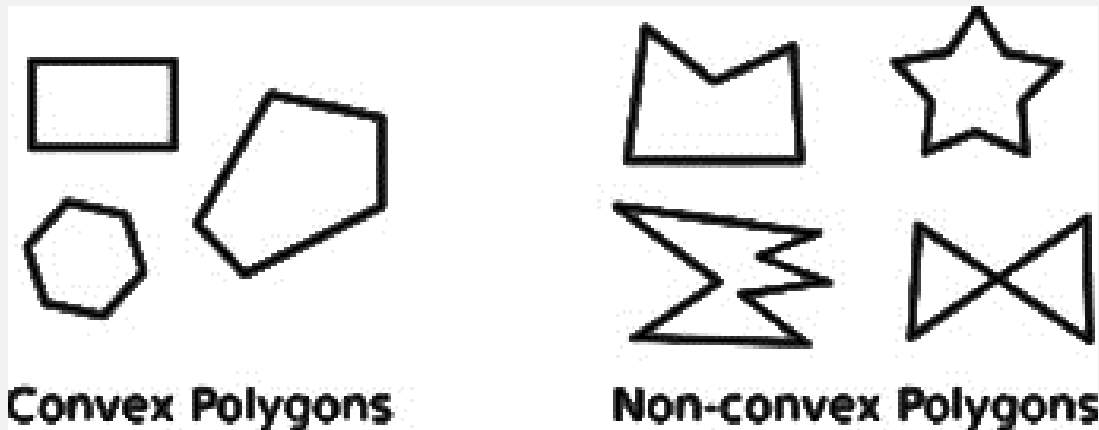
Brute Force

Convex-hull problem



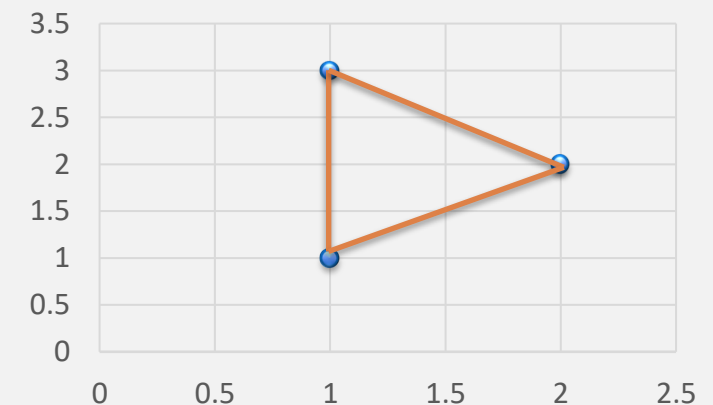
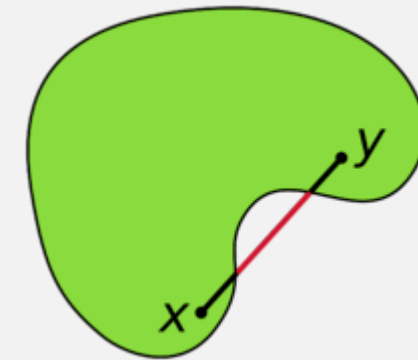
جامعة الأمير سطام بن عبدالعزيز
PRINCE SATTAM BIN ABDULAZIZ UNIVERSITY

A convex set is one in which if we connect any two points in the set, the line segment connecting these points must also be in the set.



The convex hull of two points is the line segment connecting these points.

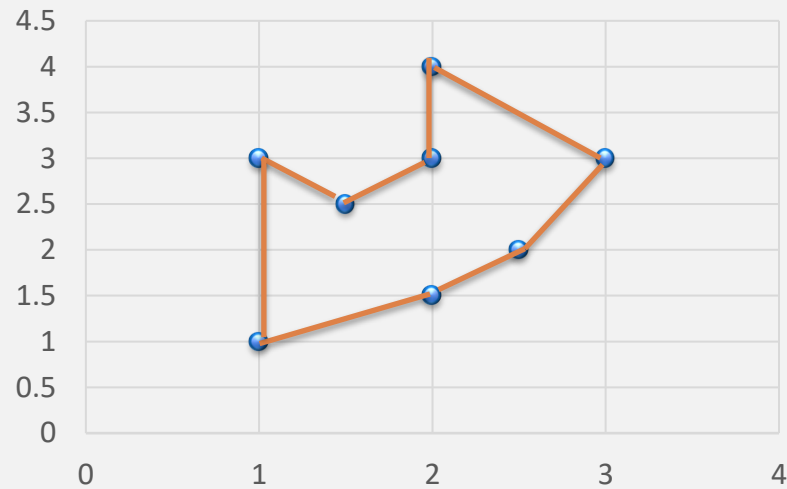
The convex hull of Three points (Not in the same line) is the triangle with the vertices at the three points.



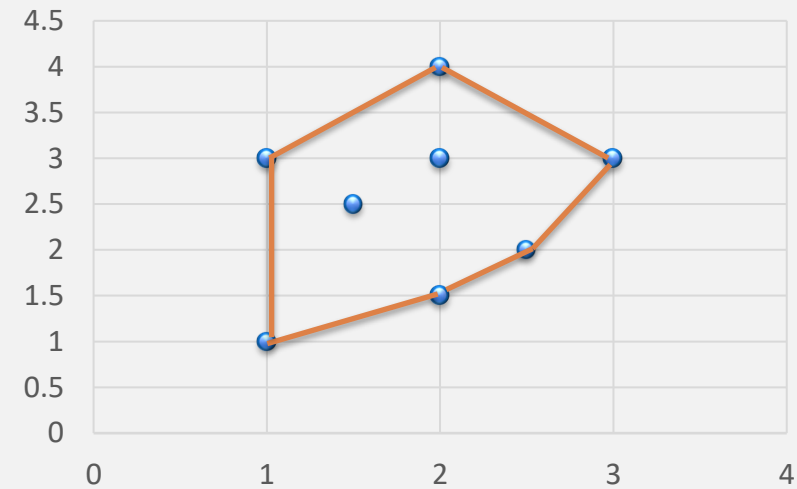
Brute Force

Convex-hull problem

Nonconvex



convex



- Formally: Find the smallest convex set containing the points.
- Informally: Find a rubber band wrapped around the "outside" points.

Brute Force

Convex-hull problem



جامعة الأمير سطام بن عبدالعزيز
PRINCE SATTAM BIN ABDULAZIZ UNIVERSITY

Convex Hull problem Brute Force Algorithm

Input: Set of point **S**

Output: Subset **C** of **S** forming the convex hull

for i in **S** do

 for j in **S** $i \neq j$ do

 inline = true

 for k in **S** $k \neq i$ and $k \neq j$ do

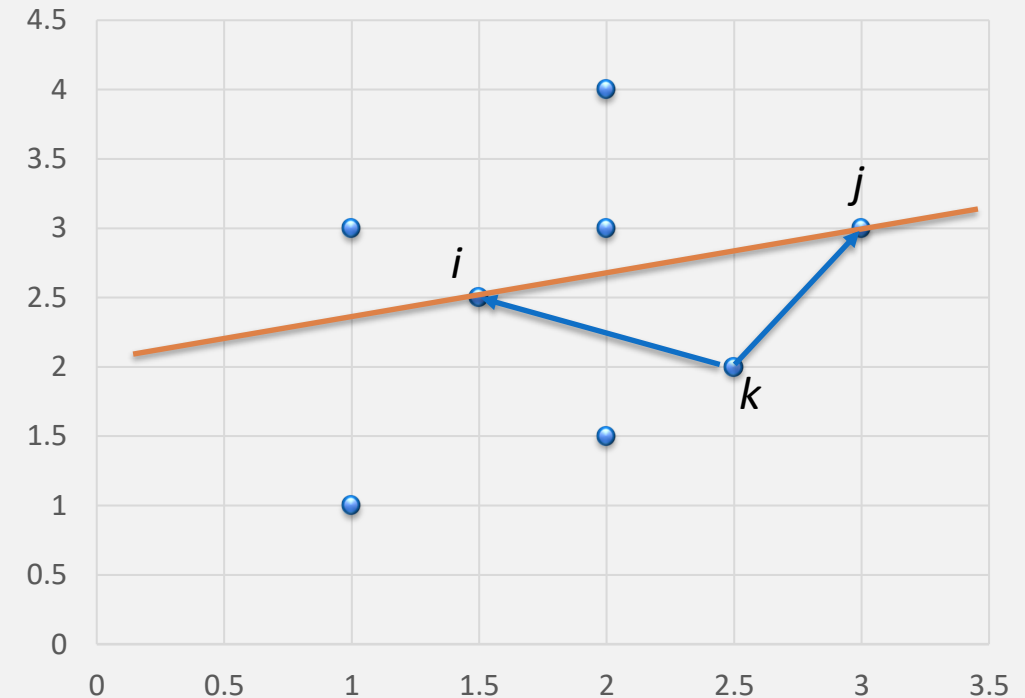
 if $(\vec{kj} \times \vec{ki} \leq 0)$

 inline = false

 break

 if (inline = true): Add i and j to **C**

return **C**



Compute the Cross Product

$$\vec{kj} \times \vec{ki} = \begin{pmatrix} (x_j - x_k) & (x_i - x_k) \\ (y_j - y_k) & (y_i - y_k) \end{pmatrix}$$

$$\vec{kj} \times \vec{ki} = (x_j - x_k) \times (y_i - y_k) - (x_i - x_k) \times (y_j - y_k)$$

Exhaustive Search

Combinatorial Problems

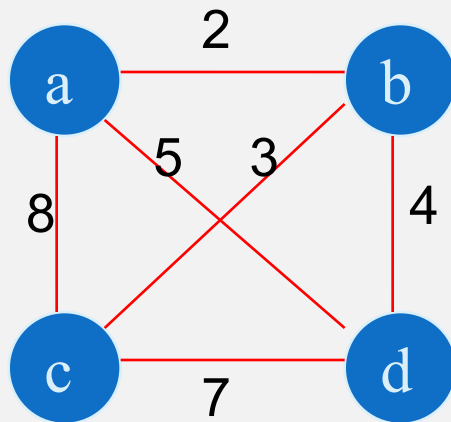
- A brute force solution to a problem involving search for an element with a special property, usually among combinatorial objects such as permutations, combinations, or subsets of a set.
- Method:
 - generate a list of all potential solutions to the problem in a systematic manner (Combinations, permutations, subsets etc.)
 - evaluate potential solutions one by one, disqualifying infeasible ones and, for an optimization problem, keeping track of the best one found so far
 - when search ends, announce the solution(s) found

Exhaustive Search

Combinatorial Problems

Traveling Salesman Problem (TSP)

- Given n cities with known distances between each pair, find the shortest tour that passes through all the cities exactly once before returning to the starting city.
- Alternatively: Find shortest *Hamiltonian circuit* in a weighted connected graph.
- Example:



Tour	Cost
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$	$2+3+7+5 = 17$
$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$	$2+4+7+8 = 21$
$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$	$8+3+4+5 = 20$
$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$	$8+7+4+2 = 21$
$a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$	$5+4+3+8 = 20$
$a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$	$5+7+3+2 = 17$

Takes $\Theta(n!)$ time to generate all possible permutation of $n-1$ cities

Exhaustive Search

Combinatorial Problems

Knapsack Problem (KP)

- There are n different items in a store
 - Item i weighs w_i pounds and is worth $\$v_i$
 - A thief breaks in
 - He can carry up to W pounds in his knapsack
 - What should he take to maximize his haul?
- Solution: Consider every possible subset of items, calculate total value and total weight and discard if more than W ; then choose remaining subset with maximum total value.
 - Takes $\Omega(2^n)$ time



Exhaustive Search

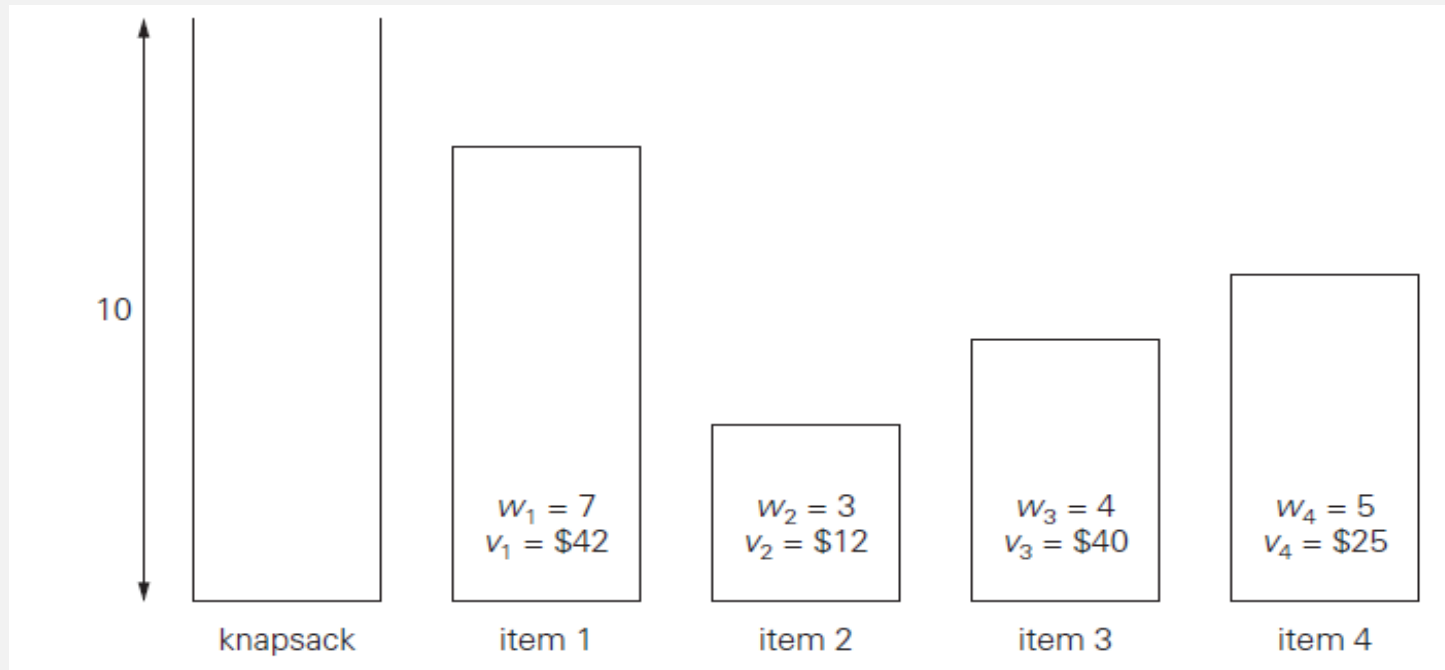
Combinatorial Problems



جامعة الأمير سطام بن عبدالعزيز
PRINCE SATTAM BIN ABDULAZIZ UNIVERSITY

Knapsack Problem (KP)

need to check 16 possibilities



Subset	Total weight	Total value
$\emptyset$	0	\$ 0
{1}	7	\$42
{2}	3	\$12
{3}	4	\$40
{4}	5	\$25
{1, 2}	10	\$54
{1, 3}	11	not feasible
{1, 4}	12	not feasible
{2, 3}	7	\$52
{2, 4}	8	\$37
{3, 4}	9	\$65
{1, 2, 3}	14	not feasible
{1, 2, 4}	15	not feasible
{1, 3, 4}	16	not feasible
{2, 3, 4}	12	not feasible
{1, 2, 3, 4}	19	not feasible

Exhaustive Search

Combinatorial Problems

The Assignment Problem

- There are n people who need to be assigned to execute n jobs, one person per job. (That is, each person is assigned to exactly one job and each job is assigned to exactly one person.) The cost that would accrue if the i th person is assigned to the j th job is a known quantity $C[i, j]$ for each pair $i, j = 1, 2, \dots, n$. The problem is to find an assignment with the minimum total cost.

	Job 1	Job 2	Job 3	Job 4
Person 1	9	2	7	8
Person 2	6	4	3	7
Person 3	5	8	1	8
Person 4	7	6	9	4

$\langle 1, 2, 3, 4 \rangle$	cost = $9 + 4 + 1 + 4 = 18$	etc.
$\langle 1, 2, 4, 3 \rangle$	cost = $9 + 4 + 8 + 9 = 30$	
$\langle 1, 3, 2, 4 \rangle$	cost = $9 + 3 + 8 + 4 = 24$	
$\langle 1, 3, 4, 2 \rangle$	cost = $9 + 3 + 8 + 6 = 26$	
$\langle 1, 4, 2, 3 \rangle$	cost = $9 + 7 + 8 + 9 = 33$	
$\langle 1, 4, 3, 2 \rangle$	cost = $9 + 7 + 1 + 6 = 23$	

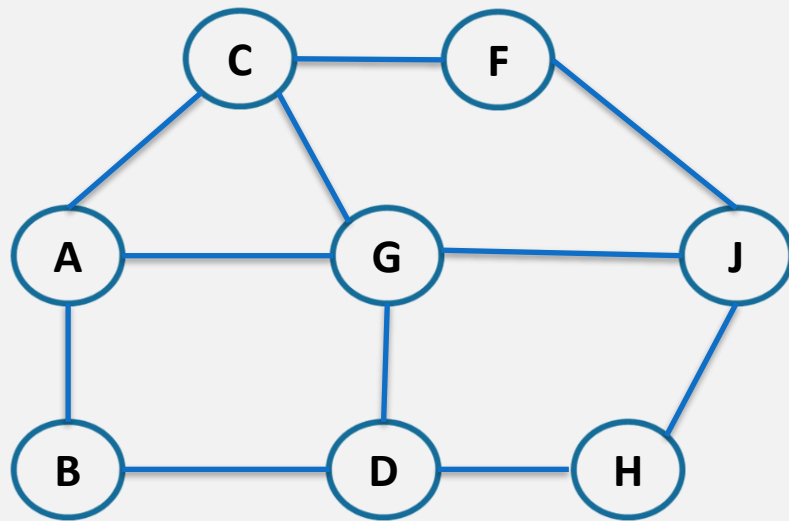
Takes $\Theta(n!)$ time to generate all possible permutation of n jobs

Exhaustive Search

Shortest Path using BFS



جامعة الأمير سطام بن عبدالعزيز
PRINCE SATTAM BIN ABDULAZIZ UNIVERSITY



Parents

Node	Parent

Queue
Front

--	--	--	--	--	--	--	--	--	--

Input: Graph **G**, Vertex **start**, Vertex **end**

Output: Path from start to end

Q.enqueue(start)

Parents[start] = null

While(Q not empty)

current = Q.dequeue()

if(current == end)

break

else

foreach neighbor of current

if(neighbor not in Parents)

Parents[neighbor] = current

Q.enqueue(neighbor)

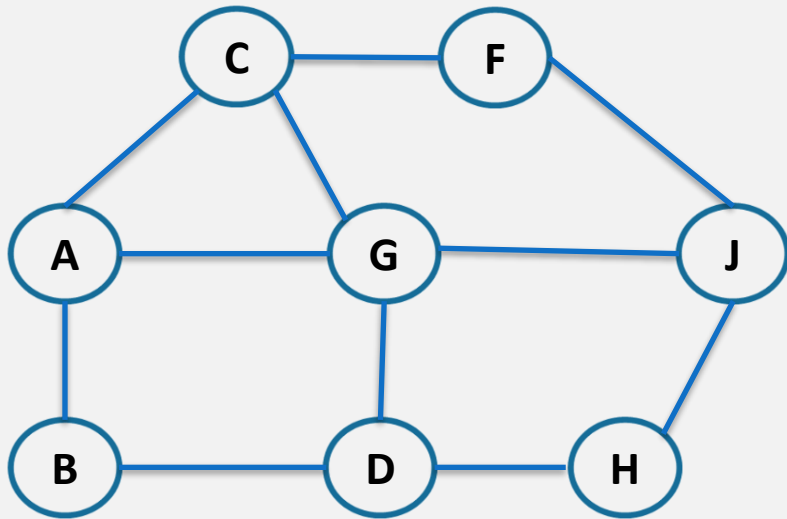
Extract the path from Parents

Exhaustive Search

Shortest Path using BFS



جامعة الأمير سطام بن عبدالعزيز
PRINCE SATTAM BIN ABDULAZIZ UNIVERSITY



The shortest path from A to J
A - G - J

Parents

Node	Parent
A	--
B	A
C	A
G	A
D	B
F	C
J	G
H	D

Queue
Front

A	B	C	G	D	F	J	H
---	---	---	---	---	---	---	---

Input: Graph **G**, Vertex **start**, Vertex **end**

Output: Path from start to end

Q.enqueue(start)

Parents[start] = null

While(Q not empty)

current = Q.dequeue()

if(current == end)

break

else

foreach neighbor of current

if(neighbor not in Parents)

Parents[neighbor] = current

Q.enqueue(neighbor)

Extract the path from Parents